

Optimization(优化)

gradient descent(梯度下降)

Question:类神经网络训练不起来

gradient接近0的点称为critical point，可能有以下两种情况导致失败：

1. local minima(局部最小值)
2. saddle point(鞍点)

saddle point不是我们想要的点，需要想办法逃离

区分办法1：泰勒展开近似，利用海森矩阵

找到 $L(\theta)$ around $\theta = \theta'$ can be approximated below

$$L(\theta) \approx L(\theta') + (\theta - \theta')^T g + \frac{1}{2}(\theta - \theta')^T H(\theta - \theta')$$

Gradient g is a vector:

$$g = \nabla L(\theta')$$
$$g_i = \frac{\partial L(\theta')}{\partial \theta_i}$$

Hessian H is a matrix:

$$H_{ij} = \frac{\partial^2 L(\theta')}{\partial \theta_i \partial \theta_j}$$

$$L(\theta) \approx L(\theta') + (\theta - \theta')^T g + \frac{1}{2}(\theta - \theta')^T H(\theta - \theta')$$

∴ critical point: 第二项=0，通过第三项来区分是local min, local max还是saddle point

$$L(\theta) \approx L(\theta') + \frac{1}{2}(\theta - \theta')^T H(\theta - \theta')$$

将 $(\theta - \theta')^T H(\theta - \theta')$ 简写作 $v^T H v$

- For all v : $v^T H v > 0 \Rightarrow L(\theta) > L(\theta') \Rightarrow$ local minima
- For all v : $v^T H v < 0 \Rightarrow L(\theta) < L(\theta') \Rightarrow$ local maxima
- sometimes $v^T H v > 0$, sometimes $v^T H v < 0 \Rightarrow$ saddle point

即 H 正定时为local minima， H 负定时为local maxima， H 不定时为saddle point

在saddle point处，根据需求选eigenvalue(特征值)找出对应的eigenvector(特征向量)进行优化：

1. 特征值 > 0 : 向着该方向可找最大值
2. 特征值 < 0 : 向着该方向可找最小值

统计表明, 鞍点数量远远多于局部最小值

区分方法2: Batch(BGD批量梯度下降)

$$\theta^* = \arg \min_{\theta} L$$

Review: Optimization with Batch

$$\theta^* = \arg \min_{\theta} L$$

➤ (Randomly) Pick initial values θ^0

➤ Compute gradient $\mathbf{g}^0 = \nabla L^1(\theta^0)$

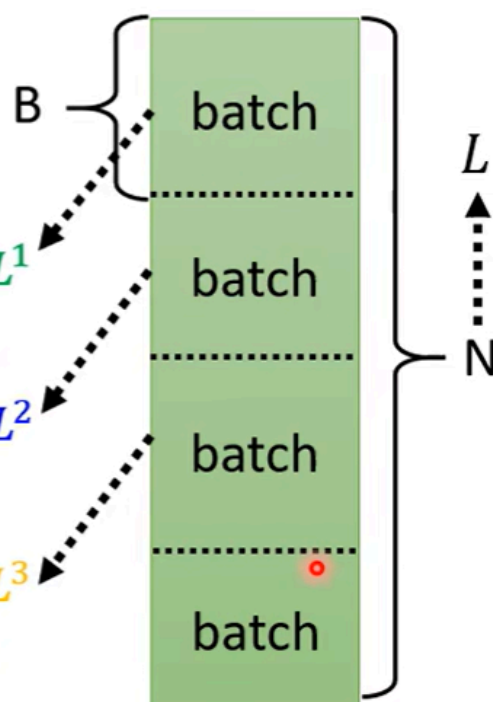
$$\text{update } \theta^1 \leftarrow \theta^0 - \eta \mathbf{g}^0$$

➤ Compute gradient $\mathbf{g}^1 = \nabla L^2(\theta^1)$

$$\text{update } \theta^2 \leftarrow \theta^1 - \eta \mathbf{g}^1$$

➤ Compute gradient $\mathbf{g}^3 = \nabla L^3(\theta^2)$

$$\text{update } \theta^3 \leftarrow \theta^2 - \eta \mathbf{g}^3$$



1 epoch = see all the batches once → Shuffle after each epoch

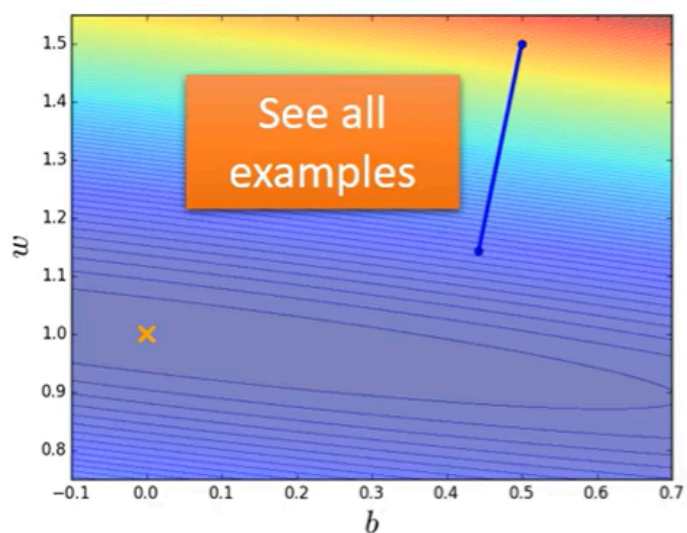
在每一个epoch开始前进行一次shuffle(数据打乱), 使每次的batch都分得不一样

Small Batch v.s. Large Batch

Consider 20 examples ($N=20$)

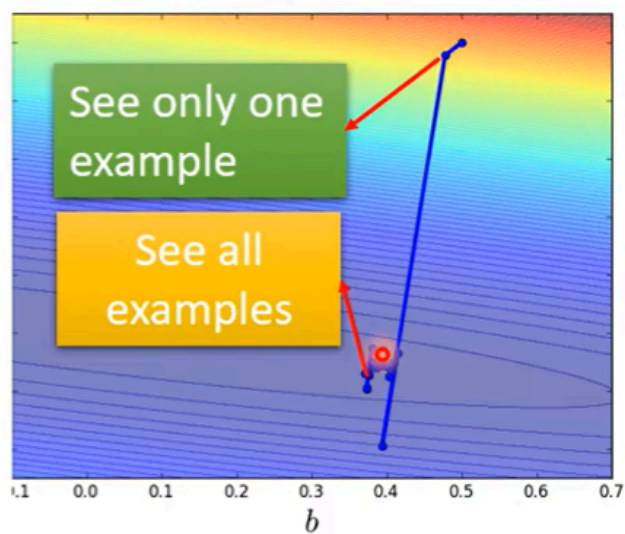
Batch size = N (Full batch)

Update after seeing all
the 20 examples



Batch size = 1

Update for each example
Update 20 times in an epoch



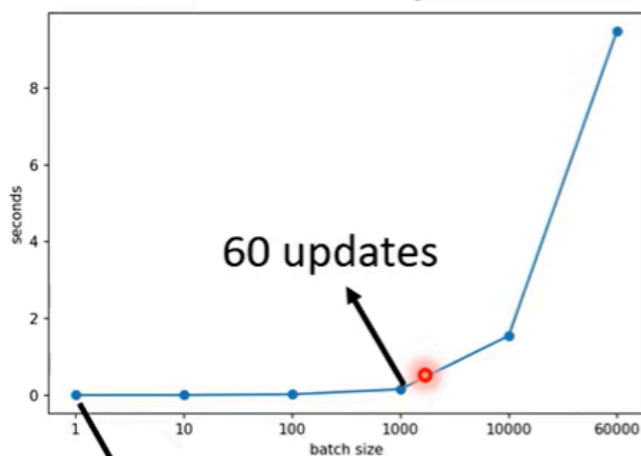
当不使用batch(左边)时, 需把所有数据都看完才会update一次: 蓄力久但powerful

当使用batch(右边)时，看完一个batch就update一次：蓄力短但noisy

Small Batch v.s. Large Batch

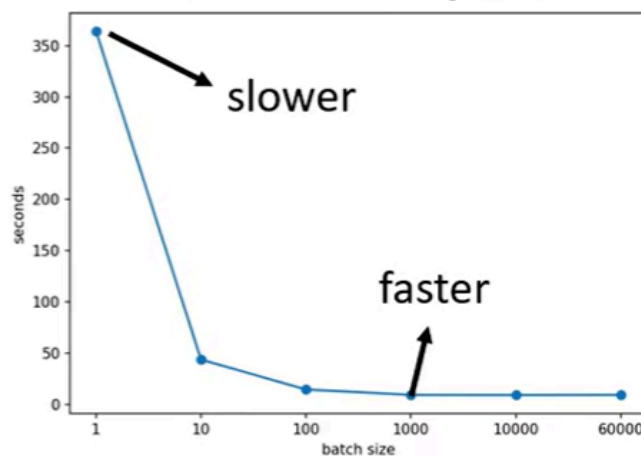
- Smaller batch requires longer time for one epoch (longer time for seeing all data once)

Time for one **update**



60000 updates in one epoch

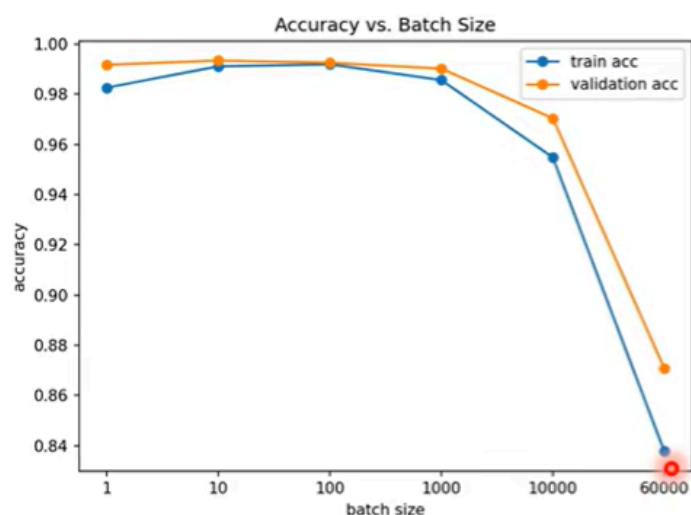
Time for one **epoch**



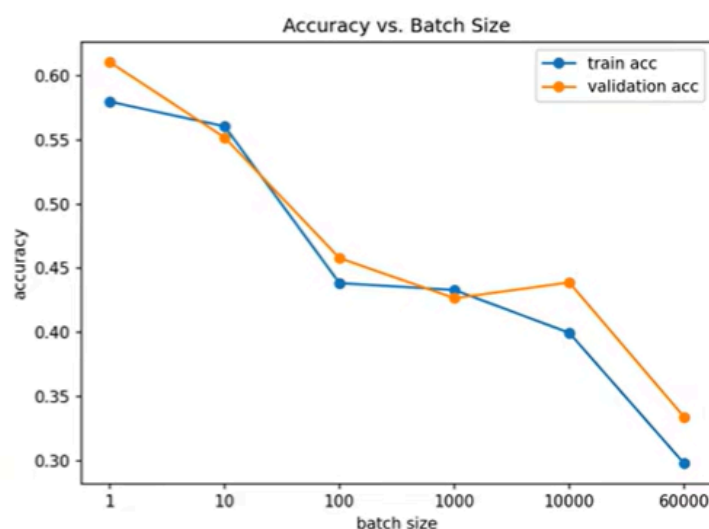
由于GPU的存在，可进行平行运算，比较大的batch更有效率

Small Batch v.s. Large Batch

MNIST



CIFAR-10



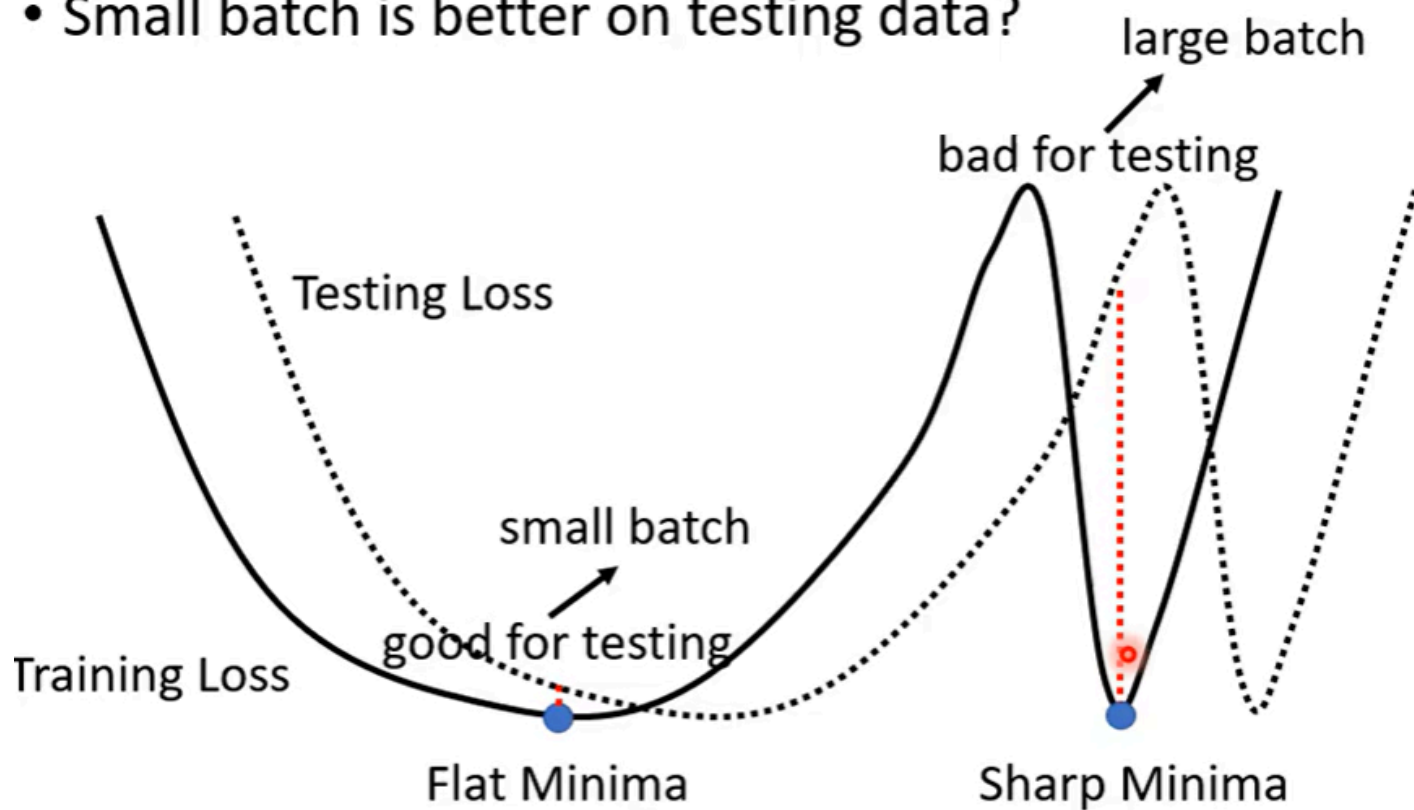
- Smaller batch size has better performance
- What's wrong with large batch size? Optimization Fails

然而事实证明，当batch较大时，效果会变差（此处不是overfitting）

当batch较小时，由于noisy存在，可能使update跨过在某一个batch上 $g = 0$ 的点

进一步的研究发现即使在train时小的和大的效果差不多，但是大的batch在testig时效果不如小的batch(overfitting)

- Small batch is better on testing data?

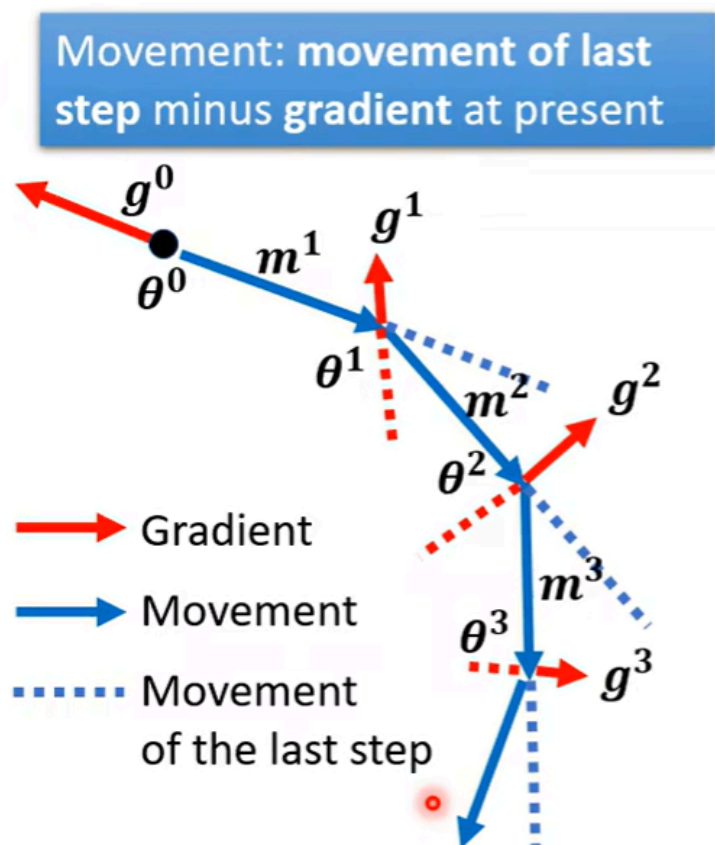


由于training data使随机取样，所以当有微小偏差的时候，在峡谷中的minima Loss更大，而在盆地中的minima Loss较小
小的batch倾向于走进盆地minima，大的batch倾向于让人走进峡谷minima

区分方法3: momentum(动量)

在GD中加入物理中的惯性

Gradient Descent + Momentum



Starting at θ^0

Movement $m^0 = 0$

Compute gradient g^0

Movement $m^1 = \lambda m^0 - \eta g^0$

Move to $\theta^1 = \theta^0 + m^1$

Compute gradient g^1

Movement $m^2 = \lambda m^1 - \eta g^1$

Move to $\theta^2 = \theta^1 + m^2$

Movement not just based on gradient, but previous movement.

在做GD时，向着 g 反方向和上一步移动方向的合方向移动

$\therefore m^i$ 为 $g^0, g^1, \dots, g^{i-1}$ 的和

$$\begin{aligned} m^0 &= 0 \\ m^1 &= -\eta g^0 \\ m^2 &= -\lambda \eta g^0 - \eta g^1 \\ &\dots \end{aligned}$$

λ 是人为设定的参数，一般=0.9